

You can break out of this execution by pressing *Ctrl-C* at the GHCi prompt.

The `openFile`, `hGetContents`, `hClose` functions can be used to obtain a handle for a file, to retrieve the file contents, and to close the handle respectively. This is similar to file handling in C. Their type signatures are shown below:

```
ghci> :m System.IO

ghci> :t openFile
openFile :: FilePath -> IOMode -> IO Handle

ghci> :t hGetContents
hGetContents :: Handle -> IO String

ghci> :t hClose
hClose :: Handle -> IO ()
```

The different IO modes are `ReadMode`, `WriteMode`, `AppendMode` and `Read-WriteMode`. They are defined as follows:

```
-- | See 'System.IO.openFile'
data IOMode = ReadMode | WriteMode | AppendMode |
ReadWriteMode
deriving (Eq, Ord, Ix, Enum, Read, Show)
```

An example code is illustrated below:

```
import System.IO

main = do
  f <- openFile "/etc/resolv.conf" ReadMode
  text <- hGetContents f
  putStr text
  hClose f
```

Executing the code in GHCi produces the following output:

```
ghci> main

# Generated by NetworkManager
nameserver 192.168 .1 .1
```

A temporary file can be created using the `openTempFile` function. It takes as input a directory location, and a pattern string for the file name. Its type signature is as follows:

```
ghci> :t openTempFile
openTempFile :: FilePath -> String -> IO (FilePath, Handle)
```

An example is shown below:

```
import System.IO
import System.Directory (removeFile)

main = do
  (f, handle) <- openTempFile "/tmp" "abc"
  putStrLn f
  removeFile f
  hClose handle
```

You must ensure that you remove the file after using it. An example is given below:

```
ghci> main
/tmp/abc2731
```

The operations on opening a file to get a handle, getting the contents and closing the handle can be abstracted to a higher level. The `readFile` and `writeFile` functions can be used for this purpose. Their type signatures are as follows:

```
ghci> :t readFile
readFile :: FilePath -> IO String

ghci> :t writeFile
writeFile :: FilePath -> String -> IO ()
```

The `/etc/resolv.conf` file is read and written to `/tmp/resolv.conf` in the following example:

```
main = do
  text <- readFile "/etc/resolv.conf"
  writeFile "/tmp/resolv.conf" text
```

You can also append to a file using the `appendFile` function:

```
ghci> :t appendFile
appendFile :: FilePath -> String -> IO ()
```

An example is shown below:

```
main = do
  appendFile "/tmp/log.txt" "1"
  appendFile "/tmp/log.txt" "2"
  appendFile "/tmp/log.txt" "3"
```

The content of `/tmp/log.txt` is '123'.

The actual definitions of `readFile`, `writeFile` and `appendFile` are in the `System.IO` module in the Haskell base package:

```
readFile :: FilePath -> IO String
readFile name = openFile name ReadMode >>= hGetContents

writeFile :: FilePath -> String -> IO()
writeFile f txt = withFile f WriteMode \ hdl-> hPutStr hdl txt
```